URI Test Case Log

2026-04-23 17:08:59 MDT

Read the task prompt in `prompts/test_case_uri.md` and confirmed the required outputs:

- `docs/test_case_uri_log.md`
- `docs/uri_test_case.md`
- a saved Python script in `scripts/` if I used more than a few Python commands

Initial thought: focus on live server behavior first, because the project is about inferring meaning from imperfect metadata and this server may itself be inconsistent.

Commands:

```
sed -n '1,220p' prompts/test_case_uri.md
git status --short
rg --files
date '+%Y-%m-%d %H:%M:%S %Z'
```

2026-04-23 17:09:00 MDT

Tested live access to the URI OPeNDAP server. The sandbox could not resolve the host, so I retried with escalated network access.

Commands:

```
curl -I https://sst-aqua.gso.uri.edu/opendap/
curl -L --max-time 20 https://sst-aqua.gso.uri.edu/opendap/
curl -L --max-time 20 https://sst-aqua.gso.uri.edu/opendap/catalog.xml
```

Thoughts:

- The root page is a Hyrax contents page, not a custom site.
- The root catalog is already informative semantically: names like `SST_Orbits`, `gradients_by_period`, `matchups`, `iQuamBuoy`, and `timeSeries` strongly suggest an SST/fronts workflow.
- I treated the root listing as a clue, not as truth, because root catalogs on real systems can be stale or permission-misaligned.

2026-04-23 17:10:00 MDT

Checked which top-level collections were actually reachable. This turned out to be important.

Commands:

```
curl -L --max-time 20 https://sst-
aqua.gso.uri.edu/opendap/SST_Orbits/catalog.xml
curl -L --max-time 20 https://sst-
aqua.gso.uri.edu/opendap/timeSeries/catalog.xml
curl -L --max-time 20 https://sst-
aqua.gso.uri.edu/opendap/iQuamBuoy/catalog.xml
curl -L --max-time 20 https://sst-
aqua.gso.uri.edu/opendap/JAXA_Orbits/catalog.xml
for d in JAXA_Orbits JAXA_Orbits_OLD RSS_Orbits RSS_Orbits_OLD
RSS_Orbits_REALLY_OLD SST_Orbits gradients_by_period
gradients_by_period_5day iQuamBuoy matchups matchups_v2 matchups_v3
timeSeries timeSeries_10km timeSeries_10km_old; do code=$(curl -L -o
/dev/null -s -w '%{http_code}' "https://sst-
aqua.gso.uri.edu/opendap/$d/catalog.xml"); printf '%s %s\n' "$code" "$d";
done
```

Observed access status:

- 200: SST_Orbits, gradients_by_period
- 403: JAXA_Orbits, JAXA_Orbits_OLD, RSS_Orbits, RSS_Orbits_OLD, RSS_Orbits_REALLY_OLD, gradients_by_period_5day, iQuamBuoy, matchups, matchups_v2, matchups_v3, timeSeries, timeSeries_10km, timeSeries_10km_old

Thoughts:

- This is exactly the kind of incomplete or misleading server state the broader project should expect.
- A GenAI-assisted characterization system should treat access-control behavior as metadata.
- Root catalog entries alone are not enough to determine usable content.

2026-04-23 17:11:00 MDT

Explored the accessible orbit collection in detail.

Commands:

```
curl -L --max-time 20 https://sst-
aqua.gso.uri.edu/opendap/SST_Orbits/2024/catalog.xml
curl -L --max-time 20 https://sst-
aqua.gso.uri.edu/opendap/SST_Orbits/2024/contents.html
curl -L --max-time 20 https://sst-
aqua.gso.uri.edu/opendap/SST_Orbits/2024/01/catalog.xml
curl -L --max-time 20 https://sst-
aqua.gso.uri.edu/opendap/SST_Orbits/2024/01/contents.html
```

Findings:

- SST_Orbits is organized by year, then month.

- `2024/01` contains many per-orbit NetCDF4 files named like: `AQUA_MODIS_orbit_115220_20240101T011558_L2_SST-URI_24-2.nc4`
- File sizes are typically a few hundred MB.
- The naming pattern alone suggests: `AQUA` satellite platform, `MODIS` sensor, `orbit` granularity, `L2` style product, SST theme, URI-produced or URI-conditioned output.

Thought:

- Even before reading attributes, the naming scheme is semantically rich and should be harvested by any inference system.

2026-04-23 17:12:00 MDT

Fetched DAP2 metadata for a representative orbit file.

Commands:

```
curl -L --max-time 30 https://sst-
aqua.gso.uri.edu/opendap/SST_Orbits/2024/01/AQUA_MODIS_orbit_115220_2024010
1T011558_L2_SST-URI_24-2.nc4.dds
curl -L --max-time 30 https://sst-
aqua.gso.uri.edu/opendap/SST_Orbits/2024/01/AQUA_MODIS_orbit_115220_2024010
1T011558_L2_SST-URI_24-2.nc4.das
curl -L --max-time 30 https://sst-
aqua.gso.uri.edu/opendap/SST_Orbits/2024/01/AQUA_MODIS_orbit_115220_2024010
1T011558_L2_SST-URI_24-2.nc4.info
```

Important metadata observed:

- Global title: `Conditioned 1km (L2) MODIS AQUA SST`
- Project: `SST Fronts`
- CF conventions: `CF-1.5`
- Institution: URI Graduate School of Oceanography
- Source: `Satellite observation`
- Creator: Peter Cornillon
- `cdm_data_type`: `Grid`

Important variables observed:

- `SST_In`
- `regridded_sst`
- `eastward_gradient`
- `northward_gradient`
- `qual_sst`
- `refined_mask`
- `latitude`
- `longitude`
- `time_from_start_orbit`
- `DateTime`

- nested `contributing_granules`
- nested `Regrid_to_L2eqa` with AMSR-E comparison fields

Thoughts:

- This is not just generic SST imagery. It is a conditioned orbit product prepared for front or gradient analysis.
- The presence of both SST and gradient fields, plus AMSR-E regridding, indicates a derived workflow rather than raw Level 2 distribution.

2026-04-23 17:13:00 MDT

Tried programmatic access using the requested Python environment.

Commands:

```
which -a conda micromamba mamba python python3
ls /Users/jimg/old-conda-envs
ls /Users/jimg/opt
ls /Users/jimg/opt/anaconda3/envs/opendap/bin/python
```

Thoughts:

- `conda` was not on the PATH for non-interactive shell commands.
- The requested environment does exist at `/Users/jimg/opt/anaconda3/envs/opendap/bin/python`.

2026-04-23 17:14:00 MDT

Created a reusable probe script and ran it against the live server.

Commands:

```
/Users/jimg/opt/anaconda3/envs/opendap/bin/python
scripts/explore_uri_server.py
```

Script created:

- `scripts/explore_uri_server.py`

Thoughts:

- Saving the probe as a script is better than scattering one-off Python invocations through the shell history.
- This aligns with the repo instruction to save a series of Python commands in `scripts/`.

Runtime result:

- Root catalog parsing and status checks worked.

- Metadata fetches worked.
- `pydap.open_url(...)` for the orbit dataset failed with `503 Service Unavailable`.

Interpretation:

- The server's metadata endpoints were available and responsive.
- Data-serving through pydap or the server path pydap uses was unstable at the time of the run.
- A production characterization system should distinguish metadata availability from data-subsetting availability.

2026-04-23 17:15:04 MDT

Fetched small DAP2 ASCII slices directly to validate actual values and server behavior.

Commands:

```
curl -g -L --max-time 30 'https://sst-
aqua.gso.uri.edu/opendap/SST_Orbits/2024/01/AQUA_MODIS_orbit_115220_2024010
1T011558_L2_SST-URI_24-2.nc4.ascii?SST_In%5B0:0%5D%5B0:4%5D'
curl -g -L --max-time 30 'https://sst-
aqua.gso.uri.edu/opendap/SST_Orbits/2024/01/AQUA_MODIS_orbit_115220_2024010
1T011558_L2_SST-URI_24-2.nc4.ascii?/SST_In%5B0:0%5D%5B0:4%5D'
curl -g -L --max-time 30 'https://sst-
aqua.gso.uri.edu/opendap/SST_Orbits/2024/01/AQUA_MODIS_orbit_115220_2024010
1T011558_L2_SST-URI_24-
2.nc4.ascii?/DateTime,/latitude%5B0:2%5D%5B0:4%5D,/longitude%5B0:2%5D%5B0:4
%5D,/SST_In%5B0:2%5D%5B0:4%5D,/qual_sst%5B0:2%5D%5B0:4%5D,/time_from_start_
orbit%5B0:4%5D'
curl -g -L --max-time 30 'https://sst-
aqua.gso.uri.edu/opendap/SST_Orbits/2024/01/AQUA_MODIS_orbit_115220_2024010
1T011558_L2_SST-URI_24-
2.nc4.ascii?/contributing_granules/filenames%5B0:4%5D%5B0:1%5D,/contributin
g_granules/start_time%5B0:1%5D,/contributing_granules/end_time%5B0:1%5D'
python3 - <<'PY'
import datetime
print(datetime.datetime.utcfromtimestamp(1704071758.233).isoformat()+'Z')
print(datetime.datetime.utcfromtimestamp(1704071700.62501).isoformat()+'Z')
print(datetime.datetime.utcfromtimestamp(1704072000.479).isoformat()+'Z')
print(datetime.datetime.utcfromtimestamp(1704072001.956).isoformat()+'Z')
print(datetime.datetime.utcfromtimestamp(1704072300.333).isoformat()+'Z')
PY
```

Important observations:

- Constraint expressions needed absolute variable paths like `/SST_In`, not just `SST_In`.
- Raw `SST_In` values were around `-3529` to `-3691`; with scale factor `0.005`, those correspond to about `-17.6 C` to `-18.5 C`.
- Raw latitude values around `-69652` correspond to about `-69.652 degrees`.
- Raw longitude values around `-100060` correspond to about `-100.060 degrees`.
- `DateTime = 1704071758.233` converts to `2024-01-01T01:15:58.233000Z`.

- `qual_sst` values in the small sample were all `3`.
- The first few characters of contributing granule names spell `AQUA_...`, which matches the orbit product naming.

Thoughts:

- The values are coherent with a Southern Ocean or high-latitude swath, not tropical SST.
- The scale factors and coordinate encodings are internally consistent with the DAS.
- The dataset is semantically strong enough that an agent should identify it as a conditioned MODIS Aqua SST orbit product with associated gradient information, not merely “an array with latitude/longitude.”

2026-04-23 17:16:00 MDT

Explored the second accessible collection, `gradients_by_period`.

Commands:

```
curl -L --max-time 20 https://sst-
aqua.gso.uri.edu/opendap/gradients_by_period/catalog.xml
curl -L --max-time 20 https://sst-
aqua.gso.uri.edu/opendap/gradients_by_period/contents.html
curl -L --max-time 30 https://sst-
aqua.gso.uri.edu/opendap/gradients_by_period/augmented_monthly_stats_01_202
4.nc.dds
curl -L --max-time 30 https://sst-
aqua.gso.uri.edu/opendap/gradients_by_period/augmented_monthly_stats_01_202
4.nc.das
curl -g -L --max-time 30 'https://sst-
aqua.gso.uri.edu/opendap/gradients_by_period/augmented_monthly_stats_01_202
4.nc.ascii?
day_pixel_count%5B0:2%5D%5B0:4%5D,night_pixel_count%5B0:2%5D%5B0:4%5D,day_s
um_SST%5B0:2%5D%5B0:4%5D,day_sum_magnitude_gradient%5B0:2%5D%5B0:4%5D'
curl -g -L --max-time 30 'https://sst-
aqua.gso.uri.edu/opendap/gradients_by_period/augmented_monthly_stats_01_202
4.nc.ascii?
day_pixel_count%5B150:152%5D%5B80:84%5D,night_pixel_count%5B150:152%5D%5B80
:84%5D,day_sum_SST%5B150:152%5D%5B80:84%5D,day_sum_magnitude_gradient%5B150
:152%5D%5B80:84%5D'
```

Findings:

- The catalog contains many monthly files across years, including both:
  - `monthly_stats_MM_YYYY.nc`
  - `augmented_monthly_stats_MM_YYYY.nc`
- The representative augmented dataset is a `360 x 180` lon-lat grid.
- Variables include:
  - `day_pixel_count`, `night_pixel_count`
  - `day_sum_SST`, `night_sum_SST`
  - `day_sum_eastward_gradient`, `day_sum_northward_gradient`

    - `day_sum_magnitude_gradient`
    - matching squared-sum variables
    - alternate `*_as_*`, `*_at_*`, and `*_mag_*` fields
- The DAS was nearly empty semantically even though the variable names were descriptive.

Sample observations:

- The first sampled cells were all zero.
- A mid-grid window had large nonzero counts and sums, e.g.:
    - `day_pixel_count` around `28k` to `55k`
    - `night_pixel_count` around `43k` to `59k`
    - `day_sum_SST` around `8e5` to `1.5e6`
    - `day_sum_magnitude_gradient` around `1105` to `2328`
- One sampled `day_sum_SST` value was `-nan`, so NaN handling matters.

Thoughts:

- This collection looks like derived monthly spatial accumulations of SST and gradient statistics on a coarse global grid.
- The filenames and variable names carry most of the semantics; formal semantic metadata contributes little here.
- This is a good example of where an LLM or multimodal reasoning system could extract useful meaning from weak metadata plus structural clues.

2026-04-23 17:17:00 MDT

Final interpretation before writing `docs/uri_test_case.md`:

- The server is a mixed-quality semantic environment.
- Two collections are directly usable right now:
    - orbit-level conditioned MODIS Aqua SST products
    - monthly gradient/statistics products
- Many additional collections are advertised but forbidden.
- Metadata is strong for orbit products and weak-to-moderate for derived monthly products.
- Remote subsetting works through DAP2 ASCII when the constraint expression uses absolute variable paths.
- Programmatic data access through pydap was not reliable during this session.

This combination of rich names, strong structure, partial semantic metadata, and inconsistent accessibility is a realistic and useful test case for the overall project.

2026-04-23 17:17:16 MDT

Improved the saved Python probe so it no longer fails hard when pydap receives a `503`.

Commands:

```
/Users/jimg/opt/anaconda3/envs/opendap/bin/python
scripts/explore_uri_server.py
```

```
git status --short
```

What changed in the script:

- It still tries `pydap.open_url(...)`.
- If pydap fails, it falls back to direct DAP2 ASCII subset requests.
- The fallback requests use URL-encoded constraint expressions and preserve the server quirk that orbit variables must be addressed with absolute paths like `/SST_In`.

Verification result:

- The script completed successfully.
- It reproduced the root catalog summary.
- It reproduced the accessible-vs-forbidden collection status table.
- It reproduced the orbit metadata summary.
- It captured the pydap `503` as an operational observation instead of aborting.
- It successfully fetched ASCII fallback samples for both the orbit and monthly gradient products.

Final thought:

- The saved script is now useful as a baseline probe for other OPeNDAP servers with similar partial availability or unstable data endpoints.

2026-04-23 17:21:15 MDT

Reread the prompt and then did a second-pass analysis focused specifically on `gradients_by_period`, with the goal of inferring stronger semantic metadata and proposing COARDS-oriented guesses.

Important note:

- The prompt instructions in this repo still require `docs/uri_test_case.md`.
- There is no `docs/uri_test_case.tex` in the repo.
- The original prompt text in `prompts/original_test_case_uri.md` mentions `.tex`, but I updated the existing markdown deliverable instead because that matches the active repo instructions and current files.

Commands:

```
sed -n '1,240p' prompts/test_case_uri.md
sed -n '1,260p' docs/uri_test_case.md
sed -n '1,320p' docs/test_case_uri_log.md
rg --files docs | rg 'uri_test_case\.(tex|md)$'
rg -n
"gradients_by_period|monthly_stats|augmented_monthly_stats|day_sum_grad|day
_as_pixel_count|day_at_pixel_count" -S .
sed -n '1,260p' gradients_by_period_umm_c.json
sed -n '1,260p' original_scripts/explore_gradients.py
sed -n '1,260p' original_scripts/explore_gradients_data.py
sed -n '1,280p' original_scripts/gradients_deep_analysis.py
sed -n '1,280p' original_scripts/gradients_science_analysis.py
```

```
sed -n '1,260p' original_scripts/gradients_coords_inference.py
sed -n '1,220p' original_scripts/gradients_final_validation.py
curl -L --max-time 30 https://sst-
aqua.gso.uri.edu/opendap/gradients_by_period/monthly_stats_01_2024.nc.dds
curl -L --max-time 30 https://sst-
aqua.gso.uri.edu/opendap/gradients_by_period/monthly_stats_01_2024.nc.das
curl -L --max-time 45 https://sst-
aqua.gso.uri.edu/opendap/gradients_by_period/catalog.xml | rg -o
'augmented_monthly_stats_[0-9]{2}_[0-9]{4}\.nc|monthly_stats_[0-9]{2}_[0-9]
{4}\.nc' | sort -u | sed -n '1,20p;$p'
curl -g -L --max-time 30 'https://sst-
aqua.gso.uri.edu/opendap/gradients_by_period/augmented_monthly_stats_01_202
4.nc.ascii?
day_as_pixel_count%5B150:152%5D%5B80:84%5D,day_at_pixel_count%5B150:152%5D%
5B80:84%5D,day_sum_grad_as_per_km%5B150:152%5D%5B80:84%5D,day_sum_grad_at_p
er_km%5B150:152%5D%5B80:84%5D,day_sum_grad_mag_per_km%5B150:152%5D%5B80:84%
5D'
python3 - <<'PY'
import re, urllib.request
base='https://sst-aqua.gso.uri.edu/opendap/gradients_by_period/'
files=
['monthly_stats_01_2024.nc.dds','augmented_monthly_stats_01_2024.nc.dds']
for f in files:
    text=urllib.request.urlopen(base+f, timeout=60).read().decode()
    vars=re.findall(r'Float64 ([A-Za-z0-9_]+)\[lon = 360\]\[lat = 180\];',
text)
    print(f'FILE {f}')
    for v in vars:
        print(v)
    print('COUNT', len(vars))
    print()
base_vars=set(re.findall(r'Float64 ([A-Za-z0-9_]+)\[lon = 360\]\[lat =
180\];', urllib.request.urlopen(base+'monthly_stats_01_2024.nc.dds',
timeout=60).read().decode()))
aug_vars=set(re.findall(r'Float64 ([A-Za-z0-9_]+)\[lon = 360\]\[lat =
180\];',
urllib.request.urlopen(base+'augmented_monthly_stats_01_2024.nc.dds',
timeout=60).read().decode()))
print('AUGMENTED_ONLY')
for v in sorted(aug_vars-base_vars):
    print(v)
PY
date '+%Y-%m-%d %H:%M:%S %Z'
```

What I learned:

- `monthly_stats_*` and `augmented_monthly_stats_*` are not just naming variants. The augmented files truly add new science content.
- The original monthly file has 14 variables and is limited to day/night counts plus geographic-frame gradient sufficient statistics.
- The augmented file has 34 variables and adds:
    - SST sums and sums of squares

    - `as` and `at` count fields
    - `grad_as_per_km`, `grad_at_per_km`, and `grad_mag_per_km` sums and sums of squares
- The dimensions are already named `lon` and `lat` in the DDS, which sharply improves confidence that the file is intended as a rectilinear lon-lat grid.

Thoughts:

- The file is closer to COARDS-ready than I first said, but it still is not fully COARDS-compliant as served because the coordinate variables are missing.
- The dataset is best interpreted as a sufficient-statistics product, not a "monthly mean SST map."
- The most important missing semantic statement is not a complicated ontology term. It is a simple sentence that says the arrays hold counts, sums, and sums of squares from which monthly means and variances can be computed.

2026-04-23 17:22:00 MDT

Refined the interpretation of `as` and `at`.

Evidence:

- The source orbit product explicitly says its axes are along-scan and along-track.
- The augmented monthly variables are named `grad_as_per_km` and `grad_at_per_km`.
- The abbreviations fit "along-scan" and "along-track" more directly than alternatives like "along-swath" and "across-track."

Conclusion:

- Best guess: `as` = along-scan
- Best guess: `at` = along-track

Confidence:

- Moderate, not absolute.
- Strong enough to document as a guess, but not strong enough to present as a fact.

I corrected the prose in `docs/uri_test_case.md` accordingly instead of repeating the looser "swath/track" phrasing from some earlier local scratch scripts.

2026-04-23 17:23:00 MDT

Updated `docs/uri_test_case.md` with:

- a deeper comparison of `monthly_stats_*` vs `augmented_monthly_stats_*`
- a stronger semantic model for the dataset
- guessed global COARDS-oriented metadata
- guessed coordinate variables needed for COARDS compliance
- explicit variable-by-variable guessed metadata for all variables in the augmented file
- a plain-language interpretation of what the dataset actually is

Final thought:

- `gradients_by_period` is now a much better test case for the project than it looked at first glance, because it sits exactly in the gap between strong structure and weak semantics.
- It is interpretable, but only if we combine source-family context, variable-set comparison, dimension names, numeric behavior, and modest scientific judgment.

2026-04-23 17:39:12 MDT

Added a new section to `docs/uri_test_case.md` that answers the next prompt directly: what `gradients_by_period` is and how it was generated.

Commands:

```
date '+%Y-%m-%d %H:%M:%S %Z'
sed -n '190,380p' docs/uri_test_case.md
```

Thoughts:

- I did not do another round of live server probing for this step because the current evidence was already strong enough.
- The key facts needed for the generation narrative were already established:
  - the orbit source product is a conditioned Aqua MODIS SST product
  - the original monthly product contains only geographic-frame gradient sufficient statistics
  - the augmented monthly product adds SST and sensor-frame statistics
  - the monthly files are gridded on a `360 x 180` lon-lat structure

What I added to the writeup:

- a plain description of the data as monthly gridded sufficient statistics rather than raw observations or simple monthly means
- an inferred stepwise generation pipeline from L2 Aqua MODIS SST granules to URI/GSO-conditioned orbit products and then to monthly 1-degree accumulations
- an explicit interpretation of what “augmented” most likely means
- a concise one-paragraph product description that could serve as a high-level catalog summary

Final thought:

- This prompt sharpened an important distinction: the semantic metadata guesses are useful, but the more valuable outcome is the inferred product model.
- For `grep-dap`, that product model is probably the thing an AI assistant should try to recover first: not just variable labels, but what kind of transformation pipeline created the dataset.

2026-04-23 17:45:15 MDT

Estimated the spatial range over which the orbit-level gradients were likely calculated, and added the estimate plus reasoning to `docs/uri_test_case.md`.

Commands:

```
sed -n '1,280p' original_scripts/gradient_spatial_scale.py
sed -n '1,260p' original_scripts/gradient_spatial_scale2.py
/Users/jimg/opt/anaconda3/envs/opendap/bin/python
original_scripts/gradient_spatial_scale2.py
/Users/jimg/opt/anaconda3/envs/opendap/bin/python
scripts/estimate_gradient_spatial_range.py
```

What happened:

- The old script in `original_scripts/` failed because it uses an older `pydap` calling convention.
- I created `scripts/estimate_gradient_spatial_range.py` and first tried the same approach with the current environment.
- `pydap` again hit the server's `503 Service Unavailable` path, so I rewrote the script to use direct DAP2 ASCII subset requests instead.
- The first orbit patch had valid geolocation but no valid stored gradient values.
- I expanded the script to test multiple small orbit patches and aggregate the usable comparisons.

Measured values from the successful run:

- along-track pixel spacing mean: about `1.036 km`
- cross-track pixel spacing mean: about `1.005 km`
- finite-difference to stored-gradient ratio:
  - along-track comparisons: about `3.4` to `5.1`
  - cross-track comparisons: about `1.6` to `2.9`

Interpretation:

- The stored gradient is clearly not a simple 1-pixel nearest-neighbor derivative.
- Because the local pixel spacing is about `1 km`, the ratios imply an effective multi-pixel stencil.
- The evidence is sparse, but it supports an effective gradient range on the order of a few kilometers.
- I summarized this in the document as an estimated effective range of roughly `3–5 km`, with `about 4 km` as the best single-number estimate.

Critical note:

- This estimate is still approximate.
- The finite-difference sample count was small because valid stored gradients were sparse in the sampled orbit windows.
- I chose not to overstate the precision in the writeup.